

1. Requirements

1.1 Problem Statement

Engineering students in their 1st and 2nd year frequently encounter coding errors and conceptual gaps while learning programming languages like Python, C, and C++. The standard debugging workflow — reading a cryptic error, searching Stack Overflow, copying a fix — teaches them *what* to paste but never *why it was wrong*. This creates students who can fix bugs but not prevent them.

Problem Statement (AT-01-S4): Build an AI-based student doubt-solving chatbot that not only resolves coding and conceptual doubts but teaches the student through structured explanation.

1.2 Target Users

User Personas

Primary: 1st and 2nd year B.Tech CSE students struggling with Python, C, and C++ programming assignments.

Secondary: Any beginner-to-intermediate programmer who wants to understand errors, not just fix them.

Use context: During lab sessions, assignments, exam prep, and self-study — wherever code breaks.

1.3 Functional Requirements

- FR-01 · Accept and execute student code automatically (Python, C, C++) without manual error pasting
- FR-02 · Capture real stdout and stderr from code execution
- FR-03 · Send code + captured output to Groq AI with CRISP-structured prompt
- FR-04 · Return structured response: Error Diagnosis, Why This Happened, Fixed Code, Code Improvement, Remember This
- FR-05 · Support Concept Doubt mode for theoretical explanations with analogy and example
- FR-06 · Persist doubt history within a session; viewable in History tab
- FR-07 · Support Beginner and Intermediate levels with tone-adapted explanations

2. Testing & Resilience

Every realistic failure mode was identified and handled in code. The table below maps each edge case to its exact handling mechanism:

Scenario	How CodeFix Handles It
Empty code box submitted	st.warning() shown, execution blocked
Empty concept doubt submitted	st.warning() shown, API call blocked
Code has no errors (runs clean)	stderr shows '(no errors)', AI still gives improvement tips
Python not installed on machine	subprocess catches FileNotFoundError, returns clear message
C/C++ compiler (gcc/g++) missing	Returns 'Install MinGW / build-essential' message
Code runs > 10 seconds	subprocess timeout=10 kills process, returns TimeoutError
Groq API rate limited (429)	Auto-retry up to 3x with 20s/40s/60s wait + toast notification
Groq API timeout	requests timeout=60, caught by try/except, retried
AI response missing a section	secs.get() with '—' fallback, page still renders
Markdown code fences in response	regex strips ```lang and ``` before st.code() display

2.1 Key Resilience Patterns Used

Resilience Patterns

Input Guards: st.stop() is called immediately if required inputs are empty, preventing blank API calls.

Subprocess Isolation: Code is written to a temp file, executed in a child process, and the file is deleted after — the main app never crashes from student errors.

API Retry Loop: Groq rate limits (429) trigger an exponential wait-and-retry with user-visible toast notification instead of a hard crash.

Graceful Fallbacks: Every response section uses secs.get('Key', '—') so a missing AI section never breaks the UI.

Fence Stripping: Regex removes markdown code fences from AI output before rendering to st.code().

3. Refactoring Logs

The codebase evolved through 5 major versions over 4 weeks. Each refactor was driven either by judge feedback, discovered bugs, or UI improvements:

Version	Change Made	Why
v1.0	Basic code + error paste, Gemini API	Initial prototype
v2.0	Added CRISP prompt framework	More structured AI responses
v3.0	Added Beginner/Intermediate level selector	Adaptive explanations
v4.0	Live code execution via subprocess	Judges: remove manual error paste
v4.1	Removed manual error box	Redundant after auto-execution added

Version	Change Made	Why
v4.2	Sidebar removed, full-width layout	Cleaner UI, no duplicate history
v4.3	Switched Gemini → Groq API	Free tier, faster, no quota resets
v4.4	Response cards stacked vertically	Less cramped, easier to read
v4.5	Markdown fence stripping in parser	Fixed ```python showing as raw text
v5.0	3-tab layout: Home / Solve / History	Judges: track doubts without re-running

3.1 Architecture Shift Summary

Architecture Shift

Before (v1–v2): Single-page layout, student manually pastes code AND error, Gemini API, no history.

After (v5): 3-tab app (Home/Solve/History), code executes automatically, errors captured by subprocess, Groq API with retry, structured card breakdown, full session history.

Key structural decision: Separating the 'input view' from the 'breakdown view' using `st.session_state.breakdown` keeps the UI clean and prevents page clutter.

Prompt improvement: CRISP framework enforces structured AI responses with exact bold headings, making `parse_sections()` reliable and UI rendering consistent.